

Day 12: Normal Distribution Exhaustive Masterclass

Complete Technical Guide, Case Studies, Manual Calculations & Python Engines

1. Introduction to Normal Distribution & The Dual Engine

Normal Distribution (jise Gaussian Distribution ya Bell Curve bhi kaha jata hai) continuous random variables ka sabse mahatvapurna pillar hai. Jab real-world data (jaise products ka weight, processing time, ya measurements) bina kisi bias ke apne center point (Mean) ke aaspaas perfectly balance tarike se phaila hota hai, tab yeh sateek Bell Curve banta hai.

Gauss ka absolute vertical height nikalne ka original probability density formula do alag-alag mathematical blocks se milkar banta hai, jise hum **Dual Engine Architecture** kehte hain:

$$f(y) = [1 / (\sigma \cdot \sqrt{2\pi})] \times [e^{-0.5 \cdot ((y-\mu)/\sigma)^2}]$$

- **Component A (Maximum Base Height Engine):** $1 / (\sigma \cdot \sqrt{2\pi})$ - Yeh component curve ke perfectly center mean (μ) par absolute peak height fix karta hai.
- **Component B (The Slope Decay Factor):** $e^{-0.5 \cdot Z^2}$ - Yeh decimal factor (0 se 1 ke beech) batata hai ki center se door jaane par height kitne percent drop hogi.

2. Question 1: Quality Window Compliance Analysis (4.96g to 5.04g)

Corporate Scenario Question: Dubai Premium Gold & Diamond Park mein ek high-end precision cutting machine lagayi gayi hai jo custom luxury rings ke liye gold bars kaat-ti hai. Given parameters hain: Mean weight (μ) = 5.00g aur Standard Deviation (σ) = 0.02g.

Sawaal: Machine se banne wali kitni percent rings 4.96g se 5.04g ke exact quality bracket mein aayengi? (Empirical Rule Verification using Manual & Library Systems).

A. Manual Python Core Script (No External Libraries)

Is system mein bina kisi automatic library ke Gauss ke cumulative area matrix (Z-table lookup) ko high-precision Error Function (**erf**) ke mathematical integration se manually gootha gaya hai:

```

import math

def manual_z_score(y, mean, std_dev):
    # Z-SCORE TRANSFORM ENGINE:  $Z = (Y - \mu) / \sigma$ 
    z = (y - mean) / std_dev
    return z

def manual_z_table_lookup(z_score):
    # High-precision Error Function (erf) use karke manual cumulative area (Z-Table Grid)
    cumulative_area = 0.5 * (1 + math.erf(z_score / math.sqrt(2)))
    return cumulative_area

if __name__ == "__main__":
    MEAN_WEIGHT = 5.00    #  $\mu = 5.00g$ 
    STD_DEV = 0.02        #  $\sigma = 0.02g$ 
    lower_weight = 4.96
    upper_weight = 5.04

    # Step 1: Calculate Z-Scores Manually
    z_lower = manual_z_score(lower_weight, MEAN_WEIGHT, STD_DEV)
    z_upper = manual_z_score(upper_weight, MEAN_WEIGHT, STD_DEV)

    # Step 2: Replicate Z-Table Grid Lookups
    area_lower = manual_z_table_lookup(z_lower)
    area_upper = manual_z_table_lookup(z_upper)

    # Step 3: Interval Area Deduction (Upper Area - Lower Area)
    compliance_area = area_upper - area_lower
    compliance_percentage = compliance_area * 100
    wastage_percentage = 100.0 - compliance_percentage

    print(f"📢 COMPLIANCE RESULT : [ {compliance_percentage:.2f}% ] rings quality pass karengi.")
    print(f"⚠️ OUTLIERS / WASTAGE: [ {wastage_percentage:.2f}% ] rings reject honggi.")

```

B. Short Library Production Code

Industry mein production workflows ke dauran time bachane ke liye hum direct *scipy.stats.norm.cdf()* ka use karte hain jo automatic left-tail area calculate karta hai:

```

import scipy.stats as stats

MEAN_WEIGHT = 5.00
STD_DEV = 0.02
lower_weight = 4.96
upper_weight = 5.04

# stats.norm.cdf automatic Z-score nikal kar left-tail area de deta hai
area_lower = stats.norm.cdf(lower_weight, loc=MEAN_WEIGHT, scale=STD_DEV)
area_upper = stats.norm.cdf(upper_weight, loc=MEAN_WEIGHT, scale=STD_DEV)

compliance_percentage = (area_upper - area_lower) * 100
print(f"📢 COMPLIANCE RESULT : [ {compliance_percentage:.2f}% ] rings quality pass karengi.")

```

Dono System Ka Output Verification: Exact **95.45%** compliance milta hai, jo verify karta hai ki data sateek 2-Sigma window ke andar aata hai, aur baki **4.55%** wastage outliers mein jata hai.

3. Question 2: Gauss Height Engine Decomposition (at y = 5.02g)

Sawaal: Agar hume target point $y = 5.02g$ (center se exact 1-sigma door) par curve ki absolute vertical height (Probability Density) nikalni ho, toh Gauss ke Dual Engine (Component A & Component B) ko manually aur library shortcut se kaise execute karenge?

A. Manual Python Core Script (Dual Engine Decomposition)

```
import math

def manual_gaussian_decomposition(y, mean, std_dev):
    # COMPONENT A: MAXIMUM BASE HEIGHT ENGINE
    sqrt_two_pi = math.sqrt(2 * math.pi)
    component_a = 1.0 / (std_dev * sqrt_two_pi)

    # COMPONENT B: THE SLOPE DECAY FACTOR (% DROP)
    deviation = y - mean
    z_score = deviation / std_dev
    exponent_power = -0.5 * (z_score ** 2)
    component_b = math.exp(exponent_power)

    # FINAL PROFILE HEIGHT ASSEMBLY
    final_profile_height = component_a * component_b
    return component_a, component_b, final_profile_height

if __name__ == "__main__":
    comp_a, comp_b, final_height = manual_gaussian_decomposition(5.02, 5.00, 0.02)
    print(f"Component A (Base Max Height): {comp_a:.4f}")
    print(f"Component B (Decay Percentage): {comp_b:.4f}")
    print(f"📢 ABSOLUTE VERTICAL HEIGHT f(5.02) = {final_height:.2f}")
```

B. Short Library Production Code

Coordinate point par sateek curve ki vertical density height nikalne ke liye library ka shortcut function **stats.norm.pdf()** (Probability Density Function) use hota hai:

```
import scipy.stats as stats

MEAN_WEIGHT = 5.00
STD_DEV = 0.02
target_y = 5.02

final_height = stats.norm.pdf(target_y, loc=MEAN_WEIGHT, scale=STD_DEV)
print(f"📢 ABSOLUTE VERTICAL HEIGHT f(5.02) = {final_height:.2f}")
```

Dono System Ka Output Verification: Dono tariko se vertical density height perfectly **12.10** calculate hoti hai. Component B (0.6065) batata hai ki center peak se 1-sigma door jaane par height apni mool height ki **60.65%** bachi hai.